Technical Writeup

AI-powered employee onboarding | Prototype and production direction

Summary

A Workday "hire completed" event triggers an n8n workflow. A configurable LLM proposes the new hire's access; a deterministic policy engine inside an MCP server validates it; an idempotent executor provisions Okta (control plane), Google Workspace, Slack and Jira and opens FreshService tickets; every step lands in a tamper-evident audit log. Target: accounts staged within an hour of the HR event, activated on the start date, instead of 3-5 days of manual work.

Core principle: the LLM proposes, deterministic code disposes. The selected model never holds credentials, never calls target systems directly, and cannot grant anything the policy catalog doesn't allow. The planner supports Anthropic, OpenAI, and Gemini through LiteLLM; the offline mock remains the default until a model and matching API key are configured.

Design decisions and why

- n8n for orchestration, MCP server for logic. n8n is the team's primary platform, and is good at triggers, waits (approvals, activate-on-start-date) and alerting. But business logic and credentials in low-code nodes are hard to test and review, so they live in a versioned, unit-tested service exposed over MCP. n8n's AI Agent node consumes it as an MCP client. Other AI tooling (People Ops in an AI client, an on-call copilot) reuses the same governed surface instead of building one-off integrations. Zapier is not used: no durable waits or code-level error handling for this flow.
- Okta as control plane. Where Okta has SCIM/group-push to Google, Slack and Jira, the executor assigns Okta groups and lets Okta propagate (fewer credentials, one audit trail, same path leavers will use). Direct APIs only for what SCIM can't do (Slack channel membership, Jira project roles, Google OU/lists).
- Where an LLM adds real value (and where it doesn't). Deterministic rules handle the clean 80% (department to baseline). The model handles the messy parts: interpreting job titles that span teams ("Solutions Engineer, Pre-Sales"), location-aware choices (EMEA vs NA channels), username alternatives, writing the manager welcome note and Day-1 checklist, and turning raw failures into actionable tickets. Output is a forced tool call with a JSON schema, `temperature=0`, so there is nothing to parse and little variance.
- Staged accounts. Okta users are created `STAGED` and activated at 06:00 UTC on the start date, so provisioning can start days early (laptops, approvals) without live accounts sitting around.
- Data minimisation. Intake allow-lists 13 Workday fields. SSN, DOB, address and compensation never reach the selected model, logs or the MCP server.

Failures, retries, edge cases

Situation	Handling
Duplicate / replayed webhook	Dedupe on `event_id`; every write also carries an idempotency key `run:step:hash(items)` so re-running is safe
429 / 5xx / timeout	Exponential backoff + jitter, 4 attempts (transient only). Exhausted to treated as failure
403 / 404 / validation (permanent)	No retry. Continue non-critical systems, mark run `PARTIAL`, the selected model writes the incident ticket, FreshService + Slack alert
Okta create fails	Critical step: stop, run `FAILED`, ticket. Nothing downstream proceeds
Username taken	Try the model's ordered candidates; same-`worker_id` match = rehire, reuse; all taken to ticket
Missing manager / past start date / bad record	Run stops with `NEEDS_INFO`; ticket to HR. We never guess
LLM outage or invalid output	Retry once with the schema error; then fall back to the deterministic department baseline (degraded but safe), flag for human review
Prompt injection in free-text HR notes	Notes are passed in an `untrusted` block; the model is told to flag, not obey. Even if fooled, the policy gate denies (demonstrated in `run_demo.py`: the mock "manipulated" planner requests admin and is blocked)
Rescinded hire / start date moved	Workday event triggers `rollback_run` (saga compensation: revoke in reverse order, suspend Okta user) or reschedules activation
Partial failure	Connector writes use stable idempotency keys and all orchestrated connector calls are audited/retried. The MCP prototype persists run results, connector-step outcomes, approval decisions and activation targets in SQLite. Simulated target-system state remains in memory; production should use real idempotent APIs and encrypted DynamoDB/Postgres state.

Access governance and auditability

- Catalog + tiers (policy-as-code in git, Security-reviewed, versioned): tier 0 baseline (auto), tier 1 role-based (auto if within department), tier 2 sensitive (manager + app-owner approval), tier 3 privileged (never automated). Unknown ids are denied; tier 3 isn't even shown to the model.
- Least privilege everywhere: one service account per target system with only the scopes it needs, in Secrets Manager with rotation; MCP tokens are OAuth 2.1, audience-bound, scope-per-tool; mutating tools default to `dry_run`.
- Contractors: restricted entitlements, mandatory expiry (production extension).
- Audit: every decision and API call writes a record (actor, action, params with PII redacted, policy version, model id, prompt version, plan hash, verdict). Records are hash-chained and shipped to S3 Object Lock (WORM). The prototype detects tampering (see demo). The same log answers "who got what, why, and who approved it" for SOC 2 / ISO 27001 evidence.
- Human-in-the-loop: tier-2 items become FreshService approval tickets. The scoped MCP approval tool records manager and app-owner decisions from different people; after both approve, the executor applies the grant. Rejections are recorded, and an approved grant that fails can be retried without asking for approval again. People Ops sees exceptions, not every hire.

What I'd build next

- Full tool-using agent loop in n8n/MCP (prototype uses a one-shot structured plan plus an LLM triage call).
- Peer-based baselining: feed the selected model the access of the 5 most similar current employees; flag outliers (access drift).
- Connect FreshService ticket status webhooks to the approval MCP tool so managers and app owners can approve directly from the ticket; the current prototype exposes the guarded MCP decision tool but does not include a FreshService webhook adapter.
- Leaver and mover automation on the same engine; quarterly access recertification driven from the audit log.
- Eval suite for the planner (golden hires, adversarial HR notes, regression on prompt/model changes) gating deploys; cost/latency dashboards.
- Real connectors and contract tests against vendor sandboxes (Okta preview org, Slack sandbox, Atlassian sandbox).

Assumptions and how they'd change the design

- Workday can emit a Hire business-process event/webhook (else: RaaS poll every 15 min; same downstream design).
- Okta already integrates with Google, Slack and Jira via SCIM (else: more direct API calls in the executor, same policy gate).
- Slack on Enterprise Grid / Atlassian Guard (SCIM available); on lower tiers, channel/group steps use plain APIs.
- A roletoaccess matrix doesn't exist yet; the catalog in this prototype is a seed that Security + dept heads would own.
- ~600 employees so ~10-20 hires/week: no scale concerns expected; measure latency and cost for the provider/model selected for the pilot.
- Nothing here was run against live systems; connectors are simulated with fault injection.